Urban Flood Risk Platform: Project Explainer and Task Split

Demo-scale project for a 4-person team. Pilot city: Bengaluru.

1. The idea in brief

Cities like Bengaluru flood repeatedly, but most flood warnings are either too coarse (city-wide rainfall alerts) or too late (reports after the road is already under water). We want to build a platform that calculates **street-level flood risk for a city** by combining two independent sources of evidence:

- **Terrain and drainage physics (the prediction).** A digital elevation model (DEM) tells us where water will flow and pool. EPA SWMM, a drainage simulation tool, tells us which parts of the drain network overflow under a given rainfall intensity. Together they produce a *predicted* risk map for scenarios such as 50, 100 and 150 mm/hr rainfall.
- **Computer vision (the observation).** Cameras that already exist at traffic signals can be analysed by a vision model to detect standing water and roughly how deep it is. This gives us *observed* ground truth.

The platform then **fuses** the two. Where the model predicts high risk and a camera confirms water, we flag a confirmed hotspot. Where they disagree, we flag the location for review, which also tells us where the model needs improvement.

Scope of the demo. We do not have access to live traffic cameras yet. For the demo, we mock the camera data with photos taken on team members' smartphones (plus public flood images as extra test inputs). A small script replays these photos as if they were a live camera feed. The pipeline is built to accept real camera feeds later. We cover one pilot area of about 1 km² around a known low-lying spot or underpass.

What the demo shows:

1. A photo is submitted with a location and timestamp.
2. The vision model reports whether it is flooded, highlights the water, and estimates a rough depth class.
3. A map shows the predicted hazard layer for the chosen rainfall scenario, with photo points overlaid.
4. The fusion logic colours each photo point as confirmed, predicted-only, unexpected, or clear.

Honesty notes for the pitch:

- The photos are simulated stand-ins for traffic-camera frames. This is a proof of pipeline, not a validation.
- If real drain data is unavailable, the drain network is assumed and must be labelled "assumed" everywhere.
- Depth is a coarse class (ankle / knee / thigh), not centimetres.

2. Team roles at a glance

Person	Role	Main language	Owns folder
P1	Computer vision engineer	Python	`cv/`
P2	Hydrology and SWMM modeller	Python	`hydro/`
P3	GIS and data engineer	Python	`gis/`, `data/`
P4	Integration, backend and frontend lead	Python and JavaScript	`app/`

3. Repository layout and where to commit

One GitHub monorepo called `flood-demo`.

```
flood-demo/
├── data/
│   ├── photos/              # raw phone photos (P3)
│   ├── photos_meta.csv      # photo metadata (P3)
│   ├── dem/                 # DEM tiles (P3), NOT committed, see rules
│   ├── pilot.gpkg           # roads, buildings, hospitals (P3)
│   └── risk/                # hazard outputs (P2 and P3)
├── cv/                      # P1
├── hydro/                   # P2
├── gis/                     # P3
├── app/                     # P4 (backend + frontend)
│   ├── static/              # frontend files
│   └── routes/              # API endpoints
├── docs/                    # shared notes, slides, limitations
├── requirements.txt
└── README.md
```

Git rules

- **Branches:** each person works on their own branch: `p1/cv`, `p2/hydro`, `p3/gis`, `p4/app`. For larger pieces, create sub-branches such as `p1/cv-depth`.

- **Pull requests:** merge into `main` via pull request. At least one teammate reviews. Keep PRs small (one task each).
- **Commit only inside your own folder.** If you need a change in someone else's folder, ask them or open a PR that they review.
- **Commit message format:** `area: what changed`, for example `cv: add depth heuristic using pose keypoints`.
- **Do not commit large files** (DEM tiles, model weights, large photo sets). Keep these on a shared Google Drive folder and list their paths in the README. Small outputs (GeoJSON, CSV, JSON) are fine to commit.
- **Do not change the shared contracts (Section 4) without telling everyone.**

4. Shared contracts

These three interfaces let everyone work in parallel. Agree on them first.

1. CV output (P1 to P4)

```
{
  "image_id": "img_007",
  "flooded": true,
  "water_area_pct": 34.2,
  "depth_class": "knee",
  "confidence": 0.81,
  "annotated_image": "outputs/img_007_annot.png"
}
```

`depth_class` is one of `none`, `ankle`, `knee`, `thigh`, `unknown`.

2. Hazard layers (P2 and P3 to P4)

Files named `data/risk/hazard_{50|100|150}mmhr.geojson`, containing points or polygons with a `risk_score` field from 0 to 1, in EPSG:4326 (lat/lon).

3. Photo metadata (P3 to P1 and P4)

`data/photos_meta.csv` with columns: `id, filename, lat, lon, timestamp, camera_id`.

Pilot area: choose one area of about 1 km² on the first day (for example around the Silk Board, Hebbal or KR Circle underpass). Everyone needs this fixed before starting.

5. Tasks by person

P1: Computer vision engineer

Language: Python **Branch:** `p1/cv` **Commit to:** `cv/`, plus the evaluation report to `docs/`

What each part of the stack does

Tool	Its job in this project
PyTorch	Deep learning framework used for training and running the water segmentation model
Hugging Face `transformers` (SegFormer-b0)	Provides the pretrained segmentation model we fine-tune to label each pixel as water or background
`ultralytics` (YOLOv8-pose)	Detects people and their body keypoints (ankle, knee) so we can judge how high the water reaches
OpenCV	Reads images, resizes them, draws the water overlay, saves annotated images
Albumentations	Data augmentation (brightness changes, blur, rain-like noise) so the model copes with different conditions
Google Colab	Free GPU for training, since fine-tuning on a laptop CPU is too slow

Tasks

- [ ] Download the road-level flood datasets (Kaggle "Flood Area Segmentation" and Roboflow Universe road-flood sets). Note: FloodNet is aerial drone imagery and does not match street-level photos, so do not rely on it. Commit: nothing large; document dataset links in `cv/README.md`.
- [ ] Write the dataset loader and augmentation pipeline. Commit: `cv/dataset.py`.
- [ ] Fine-tune SegFormer-b0 for 2 classes (water, background) on Colab and track IoU. Commit: `cv/train_seg.py` (weights stay on Google Drive, path listed in README).
- [ ] Label 15 to 30 of the team's own phone photos for water, and add them to the fine-tuning or test set. Commit: label files under `cv/labels/` if small.
- [ ] Write the depth heuristic: run YOLOv8-pose to find people, read ankle and knee y-coordinates, check where the water mask boundary falls relative to them, and map to `ankle / knee / thigh / none`. Return `unknown` when no person is visible. Commit:

`cv/depth.py`.

- [ ] Write the main function `analyze(image_path) -> dict` that returns the contract JSON and saves an annotated image with the mask overlay. Commit: `cv/analyze.py`.
- [ ] Test on all team photos and build a small table of true versus predicted flooded. Specifically test wet-road reflections and puddle shadows, which are the common false positives. Commit: `docs/cv_eval_report.md`.
- [ ] Support P4 with integration, and write the limitations slide. Commit: `docs/limitations.md`.

Done when: `python -m cv.analyze data/photos/img_001.jpg` prints valid JSON and saves an annotated image in under 3 seconds on CPU.

P2: Hydrology and SWMM modeller

Language: Python **Branch:** `p2/hydro` **Commit to:** `hydro/`, and scenario outputs to `data/risk/`

What each part of the stack does

Tool	Its job in this project
EPA SWMM 5.2 (desktop GUI)	The drainage simulation engine. We also use the GUI to inspect the network and check results visually
`pyswmm`	Python wrapper that lets us run SWMM scenarios and read results from code
`swmmio`	Reads and edits `.inp` model files programmatically (for example, to reduce pipe capacity for the blocked-drain scenario)
pandas	Handles time series and tables of flooding results per junction
GeoPandas	Converts junction results into GeoJSON points with coordinates
matplotlib	Plots hyetographs and a chart showing flooded nodes rising with rainfall
QGIS with QSWMM plugin (optional)	Builds the `.inp` network visually from map data

Tasks

- [ ] Learn the SWMM building blocks (subcatchments, junctions, conduits, outfall, rain gauge) and build a 5-node toy model to confirm the toolchain works. Commit: `hydro/toy.inp`.
- [ ] Build the pilot-area model. Subcatchments come from P3's catchment polygons. For drains, use real data if you can get it. If not, generate a synthetic network along OSM roads with assumed 600 to 900 mm pipes and label it "assumed". Set imperviousness from land cover (about 70 to 90 percent for urban areas). Commit: `hydro/pilot.inp`.
- [ ] Write a design-storm generator (Chicago or SCS Type II hyetograph) for 50, 100 and 150 mm/hr peak intensity, 1-hour duration. Commit: `hydro/storms.py`.
- [ ] Run all three scenarios with PySWMM and extract flooding volume and hours flooded per junction. Commit: `hydro/run_scenarios.py`.
- [ ] Add a "50 percent blocked drains" scenario by reducing the capacity of selected conduits. Commit: part of `hydro/run_scenarios.py`.
- [ ] Normalise junction flooding to a 0 to 1 `risk_score` and export GeoJSON points. Commit: `data/risk/swmm_nodes_{scenario}.geojson`.
- [ ] Produce a plot showing flooded nodes increasing with rainfall intensity. Commit: `docs/swmm_scenarios.png`.
- [ ] Document all assumptions honestly (uncalibrated model, synthetic network where applicable). Commit: `hydro/ASSUMPTIONS.md`.

Done when: three scenario GeoJSONs exist and the plot shows flooded nodes increasing with rainfall.

Depends on: P3's catchment polygons and DEM.

P3: GIS and data engineer

Language: Python **Branch:** `p3/gis` **Commit to:** `gis/` for code, `data/` for small outputs (CSV, GeoJSON, GeoPackage)

What each part of the stack does

Tool	Its job in this project
`rasterio`	Reads, clips and writes raster files such as the DEM and hazard grids
`geopandas` and `shapely`	Handle vector data: roads, buildings, catchment polygons, hazard polygons
`osmnx`	Downloads roads, buildings and hospitals from OpenStreetMap for the pilot area

Tool	Its job in this project
`whitebox` (WhiteboxTools)	Terrain analysis: fills or breaches depressions, flow direction, flow accumulation, Topographic Wetness Index, catchment delineation
`exifread`	Reads GPS coordinates and timestamps from phone photo metadata
pandas	Builds and maintains `photos_meta.csv`
GeoPackage (instead of PostGIS)	Single-file spatial database, avoids database setup time
QGIS	Visual checking that all layers line up

Tasks

- [ ] Finalise the pilot area bounding box with the team, then download the Copernicus GLO-30 DEM (or SRTM) and clip it. Commit: clipping script `gis/get_dem.py` (the DEM itself stays on Google Drive).
- [ ] Pull roads, buildings and hospitals via `osmnx` into one GeoPackage. Commit: `gis/get_osm.py` and `data/pilot.gpkg` if small.
- [ ] Run terrain analysis with WhiteboxTools: depression handling, D8 flow direction, flow accumulation, depression depth, Topographic Wetness Index. Commit: `gis/terrain.py`.
- [ ] Delineate catchments and hand the polygons to P2. Commit: `data/risk/catchments.geojson`.
- [ ] Combine depression depth, TWI and flow accumulation into a weighted, normalised 0 to 1 **terrain-only hazard score**. This is the fallback if SWMM runs late. Commit: `gis/hazard.py` and `data/risk/terrain_hazard.geojson`.
- [ ] Write the photo collection protocol for the whole team: 15 to 30 geotagged photos, with a reference object or person in the frame, covering dry roads, wet roads, puddles and reflections. Commit: `docs/photo_protocol.md`.
- [ ] Write `ingest_photos.py` to read EXIF GPS and timestamps into the metadata file, with manual lat/lon entry as a fallback. Commit: `gis/ingest_photos.py` and `data/photos_meta.csv`.
- [ ] Write `mock_feed.py`, which replays photos every 5 seconds with simulated timestamps by sending them to P4′s API. Commit: `gis/mock_feed.py`.
- [ ] Build the exposure layer: flag roads and hospitals that fall inside hazard zones. Commit: `data/risk/exposure.geojson`.

- [ ] Create a QGIS project showing DEM, hotspots, catchments and photo points on one map. Commit: `docs/qgis_screenshot.png`.

Done when: the QGIS view shows the DEM, hotspots, catchments and photo points aligned.

P4: Integration, backend and frontend lead

Languages: Python (backend), JavaScript (frontend) **Branch:** `p4/app` **Commit to:** `app/`

What each part of the stack does

Tool	Its job in this project
FastAPI	The web API that receives photos, calls the CV function, and serves hazard layers and observations
Uvicorn	The server that runs the FastAPI app
Pydantic	Defines and validates the data contracts so bad data is caught early
`rasterio` and `geopandas`	Sample the hazard layers at each photo's location
SQLite via SQLAlchemy	Stores observations (photo, CV result, fusion status) with no database setup
Leaflet.js	Draws the interactive map, hazard layer and photo markers
Plain HTML, CSS and JavaScript	The frontend, with no framework to keep it simple
Streamlit with `streamlit-folium` (fallback)	Faster alternative if the custom frontend runs late
ruff	Linter that keeps code style consistent across the team

Tasks

- [ ] Set up the repo, branch protection and a ruff lint check. Commit: repo root files, `requirements.txt`, `README.md`.
- [ ] Write the Pydantic models for the three contracts. Commit: `app/models.py`.

- [ ] Create stub endpoints that return hardcoded fake data, so everyone can integrate early. Commit: `app/main.py`.
- [ ] Build `POST /observations` (accepts an image plus lat/lon, calls `cv.analyze`, saves to SQLite). Commit: `app/routes/observations.py`.
- [ ] Build `GET /observations`, `GET /risk/{scenario}` and `GET /scenarios`. Commit: `app/routes/risk.py`.
- [ ] Write the fusion logic: sample predicted risk at the photo location and apply the rule table below. Commit: `app/fusion.py`.
- [ ] Build the frontend: Leaflet map with the hazard layer, a rainfall scenario dropdown (50 / 100 / 150 mm/hr), and photo markers coloured by fusion status. Commit: `app/static/index.html`, `app/static/app.js`, `app/static/style.css`.
- [ ] Add a side panel that shows the annotated image and JSON when a marker is clicked. Commit: `app/static/app.js`.
- [ ] Poll `/observations` every 3 seconds so the map updates as the mock feed runs. Commit: `app/static/app.js`.
- [ ] Replace stubs with the real outputs from P1, P2 and P3 as they become available. Commit: updates in `app/`.
- [ ] Write the demo script, record a backup screen recording, and finish the README. Commit: `docs/demo_script.md` and `README.md`.

Fusion rule table

Predicted risk of 0.6 or more	CV says flooded	Status	Map colour
Yes	Yes	Confirmed hotspot	Red
Yes	No	Predicted, not observed	Orange
No	Yes	Unexpected flooding (model gap)	Purple
No	No	Clear	Green

Done when: `uvicorn app.main:app` is running, the page shows the map, and running `mock_feed.py` makes markers appear live.

6. Hand-offs between people

From	To	What	Where it lives
P3	P2	DEM and catchment polygons	`data/risk/catchments.geojson`, DEM on Drive
P3	P1, P4	Photo metadata	`data/photos_meta.csv`
P3	P4	Terrain hazard layer (fallback)	`data/risk/terrain_hazard.geojson`
P2	P4	Scenario hazard layers	`data/risk/swmm_nodes_{scenario}.geojson`
P1	P4	`analyze()` function	`cv/analyze.py`
P3	P4	Mock camera feed	`gis/mock_feed.py` calling P4's API

7. Tech stack summary

Layer	Tools
Computer vision	PyTorch, SegFormer-b0, YOLOv8-pose, OpenCV, Albumentations
Hydraulics	EPA SWMM 5.2, PySWMM, swmmio, pandas, GeoPandas
GIS and terrain	rasterio, GeoPandas, Shapely, osmnx, WhiteboxTools, exifread, QGIS, GeoPackage
Backend	FastAPI, Uvicorn, Pydantic, SQLite, SQLAlchemy
Frontend	Leaflet.js, HTML, CSS, JavaScript (Streamlit as fallback)
Tooling	Python 3.11, GitHub, ruff, Google Colab, Google Drive for large files